# PharmaStock AI

Pharmacy demand forecasting, stockout-risk and reorder assistant

Capstone build guide - you write every line, this guide tells you what to understand first

| | |
|---|---|
| Start | Saturday 24 October 2026 (Day 1) |
| Capstone deadline | Friday 20 November 2026 (Day 28 = submission day only) |
| Prepared | 23 October 2026 |
| Chosen data | Kaggle: Pharma sales data (milanzdravkovic) - 6 years, 8 ATC groups, one pharmacy |
| Stack | Python, pandas, scikit-learn, PostgreSQL, FastAPI, Streamlit, Git/GitHub, Render + Neon + Streamlit Community Cloud |

### How this guide works

This guide contains **no project code**. Commands appear only where you install or check tools.

- Every file has a **"Before you touch it" gate**: concepts you must be able to explain, decisions to write down, and a proof that the file works. Do not open the file until you can pass the gate.
- Come back to me at each gate. I will explain concepts with pharmacy examples, review your design, and give hints when you are stuck. I will only write code if you explicitly ask for it.

Assumed time: about 2 to 3 hours a day. Each day has three parts: **Learn** (30-45 min), **Build**, **Proof** (a visible result you can show).

### Honesty rules for the whole project

- The sales data is real (one pharmacy) but is grouped into 8 ATC product groups, not individual brands. Call them **product groups**.
- No public dataset has stock levels or lead times. Stock and lead time will be **simulated or user-entered**, labelled SIMULATED in the code, database, dashboard and report.
- Never say "predicts real stockouts". Say "stockout risk under simulated inventory".
- ML may not beat the simple baseline. Report whichever wins; that is a valid result.

### Correction to my earlier tip

Earlier I suggested creating the virtual environment with access to your global packages to save data. That makes **pip freeze** list everything on your laptop, which breaks clean deployment. Use a **plain virtual environment** and write requirements.txt by hand (Section 2). pip's download cache already stops repeat downloads.

# 1. The project in one page

| | |
|---|---|
| Problem | A small pharmacy needs to know which product groups may run out soon, how much will sell in the next week, and how much to reorder. Guessing leads to stockouts (patients go without) or overstock (money and expiry risk). |
| Users | Pharmacy owner, dispenser, or stock manager. |
| Inputs | Daily sales history (CSV). Stock on hand and supplier lead time (simulated defaults, user can edit). |
| Outputs | 7-day demand forecast per product group, stockout-risk level, reorder point, suggested reorder quantity, downloadable CSV report. |
| Prediction target | Units sold per day per product group; forecast horizon 7 days. |
| Baseline | Seasonal-naive (same weekday last week) and a moving average. The ML model must beat these on the same test windows to be called better. |
| ML candidate | One scikit-learn model using lag and calendar features (you choose after the Week 2 learning day). |
| Evaluation | Chronological split (never random), MAE as main metric, compared with the baselines on identical test periods. |

## Three different things - never mix them up

| Demand forecast | A number: how many units will probably sell on each of the next 7 days. Comes from the model. | Product group R03 will sell about 31 units a day. |
|---|---|---|
| Stockout-risk estimate | A judgement: given forecast demand, current stock and lead time, how likely is stock to run out before more arrives. Comes from inventory logic. | Stock covers 3 days but delivery takes 5 days: HIGH risk. |
| Reorder recommendation | An action: how much to order now. Comes from the reorder formulas plus your stated assumptions. | Order 120 units, rounded to pack size. |

### MVP, enhancements, out of scope

- **MVP:** load and validate data, clean and aggregate, baseline plus one ML model with evaluation, simulated inventory, stockout risk and reorder logic, PostgreSQL storage, FastAPI endpoints, Streamlit dashboard with CSV report download, tests, deployment, README.
- **Enhancements (only if MVP works by 17 Nov):** probabilistic risk bands, PDF report, trend alerts, second dataset.
- **Out of scope:** RAG, chatbot, deep learning, Docker, authentication, microservices, real pharmacy integrations, expiry tracking.

# 2. Install everything you need (end to end)

Do this on Day 1 (Sat 24 Oct). Always **check before installing**; reuse what you already have. Use Wi-Fi or a data bundle for the first install; after that most work is offline except GitHub and deployment.

## 2.1 Tools and accounts

| Item | Why | Check / get it |
|---|---|---|
| Python 3.10-3.12 | Runs everything. If you already have a version in this range, keep it. If installing fresh, 3.12 is a safe choice because libraries support it widely. | python.org/downloads - tick "Add python.exe to PATH" in the installer. |
| VS Code + Python extension | Editor, terminal, interpreter picker. | code.visualstudio.com. Install the Microsoft "Python" extension. |
| Git for Windows | Version control and pushing to GitHub. Includes the sign-in helper. | git-scm.com |
| GitHub account | Hosts your code; Streamlit Community Cloud deploys from it. | github.com |
| Kaggle account | Needed to download the dataset. | kaggle.com |
| PostgreSQL | Local database. Reuse your existing installation if psql works. | postgresql.org/download/windows (large download; skip if you already have it). |
| Render account | Hosts the FastAPI backend (free tier). | render.com - sign up with GitHub. |
| Neon account | Free cloud Postgres for the deployed app. | neon.com |
| Streamlit Community Cloud | Hosts the dashboard from your GitHub repo. | share.streamlit.io - sign in with GitHub. |

## 2.2 Check what you already have

Setup commands (PowerShell) - these install tools, they are not project code

```
python --version
git --version
code --version
psql --version
```

Anything that prints a version is installed. "Not recognized" means missing or not on PATH. Note the results in your journal.

## 2.3 Create the project folder and virtual environment

- Create the folder (for example **pharmastock**) and open it in VS Code (File, Open Folder).
- Open the terminal in VS Code (Ctrl + `). It should be PowerShell.
- Create a plain virtual environment named .venv, then activate it. You will see (.venv) at the start of the prompt.
- If activation is blocked, allow scripts for your user only (RemoteSigned), then retry.
- In VS Code: Ctrl+Shift+P, "Python: Select Interpreter", pick the one inside .venv.

Setup commands (PowerShell) - these install tools, they are not project code

```
mkdir pharmastock
cd pharmastock
python -m venv .venv
.venv\Scripts\Activate.ps1

# only if activation is blocked (run once):
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

## 2.4 Check which packages are already installed, then install only what is missing

With the environment active, ask pip about each package. Installed ones print details; missing ones print a warning. Because the environment is fresh, expect most to be missing - this check is mainly useful if you reuse an older project environment.

Setup commands (PowerShell) - these install tools, they are not project code

```
pip show pandas numpy scikit-learn matplotlib fastapi uvicorn python-multipart
pip show sqlalchemy psycopg2-binary streamlit requests httpx pytest python-dotenv
```

Install everything in **one** command so pip resolves versions once:

Setup commands (PowerShell) - these install tools, they are not project code

```
pip install pandas numpy scikit-learn matplotlib fastapi "uvicorn[standard]" python-multipart
pip install sqlalchemy psycopg2-binary streamlit requests httpx pytest python-dotenv
```

- **pandas, numpy, scikit-learn:** data handling and the ML model.
- **matplotlib:** quick charts while exploring data.
- **fastapi, uvicorn, python-multipart:** the API, the web server that runs it, and file upload support.
- **sqlalchemy, psycopg2-binary:** talking to PostgreSQL from Python.
- **streamlit:** the dashboard. **requests:** dashboard calls the API. **httpx:** needed by FastAPI's test client.
- **pytest:** automated tests. **python-dotenv:** reads a local .env file for secrets.

> **Saving internet data**
>
> - pandas, scikit-learn and streamlit are the biggest downloads. Install once; do not delete and recreate .venv unless broken.
> - If a download is cut off, rerun the same command; pip reuses its cache.
> - Do not install Docker, Jupyter, TensorFlow or PyTorch. You do not need them.
> - Download the Kaggle dataset once and keep a backup copy outside the project (for example in Documents).

## 2.5 Git and GitHub setup

- Tell Git your name and email once (use the same email as your GitHub account).
- Initialise the repository in the project folder.
- Create a **.gitignore** before your first commit. It must exclude: .venv, data/raw, .env, __pycache__, and any file containing secrets.
- Create an empty repository on github.com (no README, no licence yet), then connect your local folder to it and push. GitHub will open a browser window to sign in the first time.
- Create a **.env.example** file later that lists variable names with fake values. The real **.env** never goes to GitHub.

Setup commands (PowerShell) - these install tools, they are not project code

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git init
git status
```

## 2.6 Get the dataset and the database ready

- Dataset page: kaggle.com/datasets/milanzdravkovic/pharma-sales-data . Download, unzip into **data/raw/**. Read the licence on that page and note it (a mirror listed CC BY-NC 4.0; confirm).
- Because the licence is non-commercial with attribution, do not commit the raw files to a public repo. Put download instructions and credit in a data README instead.

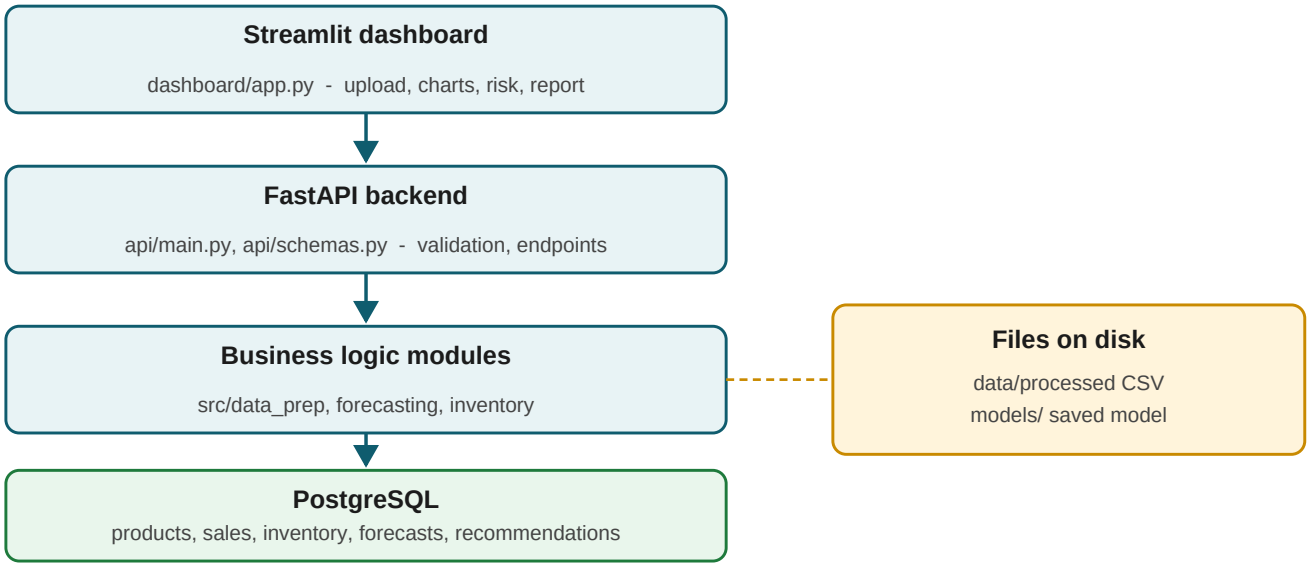
- Check Postgres: **psql --version**. If it works, create a database called pharmastock with your own username and password. Keep the password in your .env only.
- If you cannot install Postgres locally, use a free Neon database for development instead (needs internet while you work).

## 2.7 Day 1 proof

- (.venv) shows in the terminal and python can import pandas (try it in the terminal).
- GitHub shows your first commit and does **not** show data/raw or .env.
- The Kaggle files sit in data/raw on your laptop.

# 3. System architecture

A **modular monolith**: one repository, one codebase, separate folders by responsibility. The dashboard never touches the database or the model directly; it only calls the API. That keeps each part testable on its own.




## How one request travels

- User picks product group R03 on the dashboard and clicks Forecast.
- Streamlit sends an HTTP request to the API (for example GET /forecast/R03).
- FastAPI checks the input (does R03 exist? is horizon valid?) using schemas. Bad input returns a clear error message and a proper status code.
- The API calls functions in src/: load history, produce the 7-day forecast, compute risk and reorder.
- Results are saved to PostgreSQL and returned as JSON. Streamlit draws the chart and table.

## Folder structure (you create these)

Target layout

```
pharmastock/
  data/raw/          Kaggle files (not on GitHub)
  data/processed/    cleaned daily series (output of data_prep)
  models/            saved ML model file
  src/               config, inspect_data, data_prep, forecasting,
                     simulate_inventory, inventory, db
  sql/               schema.sql (table definitions)
  api/               main.py, schemas.py
  dashboard/         app.py (+ its own requirements.txt)
  tests/             test_*.py files
  notes/             journal.md, scope.md, evaluation_plan.md
  requirements.txt  .gitignore  .env.example  README.md
```

## Database tables (design, five small tables)

| Table | Purpose | Key columns (idea) |
|---|---|---|
| products | One row per product group. | id, code (e.g. R03), description, atc_group |
| daily_sales | Cleaned history. One row per product per day. | product_id, sale_date, quantity; unique on (product_id, sale_date) |
| inventory_snapshots | Stock and lead time. Flag as simulated. | product_id, snapshot_date, stock_on_hand, lead_time_days, pack_size, is_simulated |

| Table | Purpose | Key columns (idea) |
|---|---|---|
| forecasts | Predictions you made. | product_id, created_at, target_date, predicted_qty, method, model_version |
| recommendations | Risk and reorder results with assumptions. | product_id, created_at, days_of_cover, risk_level, reorder_point, reorder_qty, assumptions_text |

## API endpoints (MVP)

| Endpoint | What it does |
|---|---|
| GET /health | Returns OK. Also used to check deployment. |
| GET /products | Lists product groups and their date ranges. |
| GET /products/{code}/history | Recent daily sales for charts. |
| POST /upload | Accepts a CSV in the dataset's format, validates it, stores valid rows, returns a summary or a helpful error. |
| GET /forecast/{code} | 7-day forecast (method and model version included). |
| GET /stockout-risk/{code} | Risk level using stock and lead time. |
| GET /reorder/{code} | Reorder point, quantity and the assumptions used. |
| GET /recommendations | All groups in one list, used for the dashboard table and CSV report. |

## Dashboard pages (one Streamlit app, simple sections)

- Upload and data summary (rows, date range, missing days, warnings).
- Forecast chart: history plus next 7 days, with method label.
- Risk table with colour indicators (Low, Medium, High) and a clear SIMULATED badge.
- Reorder suggestions with the assumptions listed, and a Download report (CSV) button.

# 4. The logic you must understand before coding it

These are design recommendations and stated assumptions, not facts about your pharmacy. Write them in notes/scope.md in your own words on Day 3.

## 4.1 Data preparation ideas

- **Wide vs long:** the Kaggle daily file is expected to have one column per product group and one row per date (wide); I could not open it, so confirm this on Day 2. Databases and many tools prefer one row per product per day (long). You will decide which form each stage uses.
- **Zero vs missing:** a missing date inside a period where the pharmacy was open usually means no recorded sales, so it becomes 0 - but only after you confirm the dates are truly absent rather than the pharmacy being closed or data being lost. Write down your decision and the evidence.
- **Partial final period:** sources describe the data as ending on 8 October 2019. Decide whether to drop the incomplete tail from evaluation.
- **Fractional quantities:** values like 203.97 mean these are not clean unit counts. State this limitation and treat the unit as "sales volume".

## 4.2 Forecast evaluation

- **Chronological split:** train on the earlier period, test on the later period. Random splits leak future information (the model would see next week while learning about last week).
- **MAE:** the average absolute gap between forecast and actual, in the same units as sales. MAE of 5 means you are typically 5 units off per day.
- **Why not MAPE:** percentage errors blow up when actual sales are near zero. Mention it, do not rely on it.
- **Baseline first:** seasonal-naive says "next Monday will look like last Monday". If your ML model cannot beat that, say so honestly.
- **Leakage test:** for every feature ask, "would I know this value at the moment I make the forecast?"

## 4.3 Inventory formulas in words

| Quantity | Plain-language definition (recommended default) |
|---|---|
| Average daily demand | Mean of the 7-day forecast for that product group. |
| Days of cover | Stock on hand divided by average daily demand: how many days stock lasts. |
| Lead time demand | Average daily demand multiplied by lead time days: what sells while you wait for delivery. |
| Safety stock | A buffer for forecast error: z value (1.65 is common for about 95% service level) times the typical daily forecast error (from your test results) times the square root of lead time. State your z choice as an assumption. |
| Reorder point | Lead time demand plus safety stock. When stock falls to this level, order. |
| Stockout risk level | High if days of cover is less than lead time. Medium if cover is within a safety margin above lead time (you choose, e.g. 2 extra days). Low otherwise. Document these thresholds. |
| Reorder quantity | Forecast demand over lead time plus a review period (e.g. 7 days) plus safety stock, minus stock on hand and stock already on order; never below zero; rounded up to the pack size. |

**Simulated inventory rules (design before you build)**

- Lead time per group: pick from a small documented set such as 2, 3, 5, 7 days.
- Starting stock: a documented random multiple of recent average demand, using a fixed random seed so results can be reproduced.
- Pack size: a small documented set (e.g. 10, 20, 50).
- Every output carries is_simulated = true and the dashboard shows a SIMULATED badge. The user can overwrite stock and lead time.

# 5. Daily plan: 24 October to 20 November 2026

Rhythm: every Friday is a consolidation and review day. If you fall a day behind, absorb it on the next review day and use the cut list for that week. Do not steal time from the final week.

## Week 1 - Data and foundations (24-30 Oct)

> **Week summary**
>
> - **Deliverables:** Dataset inspected, scope written, working loader and cleaning pipeline, processed dataset, repo on GitHub.
> - **Done when:** Processed daily CSV exists and you can explain every column and every cleaning decision.
> - **Main risks:** Misreading zero vs missing days; spending too long on charts; install problems.
> - **Cut if behind:** Extra charts, README polish.

### Day 1 - Sat 24 Oct: Setup

- **Learn:** venv, Git, commits, why secrets stay out of GitHub.
- **Build:** Install and check tools, folder tree, venv, git init, GitHub repo, .gitignore, download dataset.
- **Proof:** First commit on GitHub with no data or secrets; pandas imports inside the venv.

### Day 2 - Sun 25 Oct: Inspect the data

- **Learn:** pandas basics: reading CSVs, data types, missing values, duplicates, date ranges.
- **Build:** Write a throwaway inspection script that prints columns, types, missing counts, date range, duplicates for every file. Record findings in notes/data_findings.md.
- **Proof:** You can state each file's columns, date range, and problems without looking.

### Day 3 - Mon 26 Oct: Define scope

- **Learn:** Forecast vs risk vs reorder; MVP thinking.
- **Build:** Write notes/scope.md: problem, users, inputs, outputs, target, horizon, metric, simulated-inventory statement. Send it to me for review.
- **Proof:** A one-page scope you can read aloud in two minutes.

### Day 4 - Tue 27 Oct: Loader and validation

- **Learn:** Wide vs long data; date parsing; what validation means.
- **Build:** Start src/data_prep.py: load the daily file, validate required columns, dates, numeric values, no negatives; friendly error messages.
- **Proof:** Good file passes; a deliberately broken copy fails with a clear message.

### Day 5 - Wed 28 Oct: Cleaning and calendar

- **Learn:** Zero vs missing days; building a complete date calendar.
- **Build:** Complete the calendar per product group, handle missing values, decide on the partial final period.
- **Proof:** Every group has exactly one row per day with no gaps; counts printed match your expectation.

### Day 6 - Thu 29 Oct: Processed data and EDA

- **Learn:** Seasonality: weekday and month patterns; rolling averages.
- **Build:** Save processed CSV. Make three charts: trend, weekday pattern, monthly pattern. Write five observations in your journal.
- **Proof:** Processed file reloads correctly; charts saved; observations written.

### Day 7 - Fri 30 Oct: Week 1 review

- **Learn:** Re-read your journal; explain data_prep out loud.
- **Build:** Fix leftovers, README skeleton, commit and push, review checkpoint with me.
- **Proof:** Week 1 checklist (below) fully ticked.

## Week 2 - Forecasting and inventory logic (31 Oct-6 Nov)

### Week summary

- **Deliverables:** Baselines, one ML model, honest comparison, 7-day prediction function, simulated inventory, risk and reorder functions.
- **Done when:** One call produces forecast, risk and reorder for a product group, with a written evaluation.
- **Main risks:** Data leakage; ML not beating the baseline (acceptable); formula confusion.
- **Cut if behind:** Model tuning, second ML model, probabilistic risk bands.

#### Day 8 - Sat 31 Oct: Design the evaluation

- **Learn:** Chronological split, leakage, MAE, rolling-origin testing.
- **Build:** On paper decide the test period (for example the last 12 weeks), the horizon (7), and the metric. Write notes/evaluation_plan.md before any model code.
- **Proof:** Evaluation plan written and reviewed.

#### Day 9 - Sun 1 Nov: Baselines

- **Learn:** Naive, seasonal-naive (7-day), moving average.
- **Build:** Implement baselines in src/forecasting.py.
- **Proof:** A table of baseline MAE per product group.

#### Day 10 - Mon 2 Nov: Evaluation harness

- **Learn:** Rolling-origin evaluation; fair comparison.
- **Build:** Build one reusable way to score any forecaster on the same test windows.
- **Proof:** All baselines compared in one fair table.

#### Day 11 - Tue 3 Nov: Features

- **Learn:** Lag and calendar features; why no future information.
- **Build:** Create features; check for leakage; explain each column.
- **Proof:** You can prove no feature uses future data.

#### Day 12 - Wed 4 Nov: ML model

- **Learn:** scikit-learn fit/predict; overfitting; choose one tree-based model.
- **Build:** Train one model, evaluate on the same windows as baselines.
- **Proof:** ML vs baseline MAE side by side; result recorded honestly.

#### Day 13 - Thu 5 Nov: Prediction and fallback

- **Learn:** Multi-step forecasting; saving a model; minimum-history rule.
- **Build:** Single-product 7-day prediction function; fallback to moving average when history is short; save model file; clear error for unknown groups.
- **Proof:** Prediction runs for one group; short-history and unknown-group cases behave sensibly.

#### Day 14 - Fri 6 Nov: Inventory logic

- **Learn:** Safety stock, reorder point, days of cover, reorder quantity.
- **Build:** Build simulated inventory table and the inventory functions. Hand-calculate two examples first.
- **Proof:** Function outputs match your hand calculations. Week 2 review with me.

## Week 3 - Database, API, dashboard, first deploy (7-13 Nov)

### Week summary

- **Deliverables:** Schema and data in PostgreSQL, working API, working local dashboard, health endpoint deployed.
- **Done when:** Upload a CSV and see forecast, risk, reorder and a downloaded report locally.
- **Main risks:** Postgres connection errors; API and dashboard scope creep; deployment surprises.
- **Cut if behind:** Upload of inventory CSV, extra charts, anything not in the MVP list.

### Day 15 - Sat 7 Nov: Database design

- **Learn:** Tables, primary and foreign keys, data types, basic SQL.
- **Build:** Draw the schema on paper; create the database and tables with a schema file; test the connection through an environment variable.
- **Proof:** You can connect and list the tables.

### Day 16 - Sun 8 Nov: Database functions

- **Learn:** Parameterised queries (SQL injection), transactions, avoiding duplicate inserts.
- **Build:** Write db access functions; load processed data and simulated inventory.
- **Proof:** Row counts match the CSV; running the loader twice does not duplicate rows.

### Day 17 - Mon 9 Nov: API hello and first deploy

- **Learn:** HTTP, GET/POST, status codes, FastAPI routes, auto docs at /docs, Render build and start commands.
- **Build:** Health endpoint; push; deploy only this to Render.
- **Proof:** Public /health URL opens on your phone.

### Day 18 - Tue 10 Nov: Forecast and product endpoints

- **Learn:** Pydantic schemas, response models, error handling.
- **Build:** Products, history, forecast endpoints; clear 404 for unknown group.
- **Proof:** Tested in /docs; errors are readable.

### Day 19 - Wed 11 Nov: Risk, reorder, upload

- **Learn:** File upload validation; request bodies.
- **Build:** Risk, reorder, recommendations and upload endpoints.
- **Proof:** All endpoints exercised; an invalid file is rejected with a helpful message.

### Day 20 - Thu 12 Nov: Dashboard part 1

- **Learn:** Streamlit rerun model; calling an API with requests.
- **Build:** Upload, data summary, group selector, forecast chart.
- **Proof:** Local dashboard shows a forecast fetched from the API.

### Day 21 - Fri 13 Nov: Dashboard part 2 and freeze

- **Learn:** download buttons; session state; handling a sleeping API.
- **Build:** Risk indicators, reorder table, CSV report, friendly wake-up and error messages.
- **Proof:** Full local walkthrough from upload to downloaded report. Feature freeze for MVP; Week 3 review with me.

## Week 4 - Test, deploy, document, defend (14-20 Nov)

### Week summary

- **Deliverables:** Automated tests, full deployment, README, evaluation report, demo script, defence practice.
- **Done when:** A stranger can open your deployed app or follow your README to run it locally.
- **Main risks:** Late deployment failures; documentation left to the end; cold-start delays during the demo.
- **Cut if behind:** Enhancements entirely. If deployment still fails by 18 Nov, defend with the local demo plus screenshots.

### Day 22 - Sat 14 Nov: Tests: business logic

- **Learn:** pytest; arrange-act-assert.
- **Build:** Tests for data_prep, baselines, inventory formulas.
- **Proof:** pytest passes; breaking a formula on purpose makes a test fail.

### Day 23 - Sun 15 Nov: Tests: API and edge cases

- **Learn:** FastAPI test client.
- **Build:** Tests for health, forecast, unknown product, invalid file, short history. Fix every bug found.
- **Proof:** All tests green; bug list empty.

### Day 24 - Mon 16 Nov: Full deployment

- **Learn:** Environment variables, secrets, Neon connection string, Render settings, Streamlit Cloud secrets.
- **Build:** Neon database, load data, Render API with env vars, Streamlit Cloud dashboard pointing at the API.
- **Proof:** Public URL works end to end.

### Day 25 - Tue 17 Nov: Fix and fallback

- **Learn:** Reading deploy logs.
- **Build:** Fix deployment issues; verify the local demo from a clean terminal; record screenshots and a short screen recording.
- **Proof:** Local demo starts in about five minutes; no new features from now.

### Day 26 - Wed 18 Nov: Documentation

- **Learn:** What a reviewer needs to run your project.
- **Build:** README, architecture diagram, evaluation report, limitations, API list, credit for the dataset.
- **Proof:** A classmate could run the project from the README alone.

### Day 27 - Thu 19 Nov: Defence preparation

- **Learn:** Explaining decisions honestly.
- **Build:** Write the demo script; practise the 20 questions in Section 8 twice; final secret scan of the repo.
- **Proof:** A timed 7-minute demo.

### Day 28 - Fri 20 Nov: Submission day

- **Learn:** Nothing new.
- **Build:** Final checks of links, repo and deployed app. Submit early in the day.
- **Proof:** Submitted. Links open.

> **Daily routine (about 2.5 hours)**
>
> - 30-45 min Learn: read, then explain the idea in your own words in notes/journal.md.
> - 60-90 min Build: write the code yourself in small pieces. Run it after every few lines.
> - 15 min Proof: produce the visible result and test one failure case.
> - 5 min Git: commit with a message that says what you made; push. Note questions for me.

# 6. Before you touch each file

For every file: understand the **Before** list, write down the **Decide** answers, then build, then pass the **Proof**. If you cannot answer a Before question, stop and ask me.

### **notes/scope.md and notes/journal.md** - Day 3 and daily

**Purpose:** Your written thinking: what the project is and what you learned.

**Before you touch it, be able to explain:**

- What problem a pharmacy faces with stockouts and overstock.
- The difference between forecast, risk and reorder.

**Decide and write down first:**

- Problem statement, users, MVP list, out-of-scope list.
- The exact sentence you will use to describe simulated inventory.

**Proof it works:** A one-page scope you can explain without reading.

### **.gitignore, .env.example, requirements.txt** - Day 1 and Day 24

**Purpose:** Keeps secrets out of GitHub and lets others (and Render) rebuild your environment.

**Before you touch it, be able to explain:**

- What a secret is and why a public repo exposes it forever.
- What requirements.txt does and why it should list only packages you use.
- Why file names matter on Linux (Render) even when Windows ignores capital letters.

**Decide and write down first:**

- Which folders are never committed.
- Which packages and which versions the API needs vs the dashboard.

**Proof it works:** A fresh clone plus pip install -r requirements.txt runs the API.

### **src/config.py** - Day 4

**Purpose:** One place for paths and environment variables.

**Before you touch it, be able to explain:**

- What an environment variable is and how Python reads it.
- Why the database URL is never typed into code.
- Difference between a local .env file and variables set on Render.

**Decide and write down first:**

- Names of variables (for example DATABASE_URL, API_URL).
- Where data and model paths are defined.

**Proof it works:** Changing a variable changes behaviour without editing code elsewhere.

### **src/inspect_data.py (throwaway)** - Day 2

**Purpose:** Look at the real data before designing anything.

**Before you touch it, be able to explain:**

- How to read a CSV, list columns, data types, missing values, duplicates, date range.
- What each Kaggle file (daily, weekly, monthly, hourly) represents.

**Decide and write down first:**

- Which file you will use as the main source and why.
- Anything strange: gaps, negatives, partial periods, odd values.

**Proof it works:** notes/data_findings.md lists the real column names and problems. Do not assume column names - read them.

### **src/data_prep.py** - Days 4-6

**Purpose:** Turns raw CSV into a clean daily series per product group.

**Before you touch it, be able to explain:**

- Wide vs long layout; date parsing; building a full calendar.
- Zero vs missing days and how you will prove which is which.
- Validation: what counts as an invalid file and what message the user should see.

**Decide and write down first:**

- Handling of the incomplete 2019 tail.
- Name and order of output columns; processed file location.
- How negative or non-numeric values are treated.

**Proof it works:** Processed CSV reloads; every group has one row per day; a broken file gives a clear error.

## **src/forecasting.py** - Days 9-13

**Purpose:** Baselines, features, ML model, evaluation, 7-day prediction.

**Before you touch it, be able to explain:**

- Why random splits leak the future; what a chronological split is.
- How seasonal-naive and moving average work.
- What lag and calendar features are, and why each is safe.
- What MAE means in sales units.
- Predicting 7 days ahead: why you cannot use unknown future sales as input and how recursive forecasting handles it.
- How to save and reload a model file, and why the saved file must match the library version.

**Decide and write down first:**

- Test period and horizon.
- Minimum history needed before using ML (below that use the moving average).
- Which one ML model you will try, and why.

**Proof it works:** A table comparing baseline MAE and ML MAE on identical windows; a 7-day forecast for one group.

## **src/simulate_inventory.py** - Day 14

**Purpose:** Creates the clearly labelled simulated stock and lead-time table.

**Before you touch it, be able to explain:**

- Why simulation is acceptable only when labelled.
- What a random seed is and why it makes results repeatable.

**Decide and write down first:**

- Allowed lead times and pack sizes.
- How starting stock relates to average demand.

**Proof it works:** Same seed gives the same table; the SIMULATED flag is present.

## **src/inventory.py** - Day 14

**Purpose:** Days of cover, risk level, reorder point, reorder quantity.

**Before you touch it, be able to explain:**

- Every formula in Section 4.3 in your own words.
- What happens when stock is 0, demand is 0, or lead time is missing.

**Decide and write down first:**

- Risk thresholds; z value; review period; rounding to pack size.
- What the function returns when inputs are missing (explain, do not guess).

**Proof it works:** Two hand-worked examples match the function; edge cases do not crash.

## **sql/schema.sql and src/db.py** - Days 15-16

**Purpose:** Creates tables and talks to PostgreSQL safely.

**Before you touch it, be able to explain:**

- Primary key, foreign key, unique constraint, data types.
- Why queries must use parameters instead of pasting values into text (SQL injection).
- What a transaction is.
- How to avoid duplicate rows when loading twice.

**Decide and write down first:**

- Final table list and columns.
- Which operations are inserts and which are updates.

**Proof it works:** Row counts in the database match the CSV; second load changes nothing.

## **api/schemas.py** - Day 18

**Purpose:** Defines the shape of requests and responses.

**Before you touch it, be able to explain:**

- What validation does automatically and what it cannot catch.
- Why response shapes should be fixed and documented.

**Decide and write down first:**

- Field names and types for each response; error response format.

**Proof it works:** Wrong input gives a clear 4xx error; /docs shows the schemas.

## **api/main.py** - Days 17-19

**Purpose:** All endpoints and error handling.

**Before you touch it, be able to explain:**

- GET vs POST; path vs query parameters; common status codes (200, 400, 404, 422, 500).
- How uvicorn runs the app and how Render will start it.
- How file upload works and why files must be validated.
- Why startup should not retrain models.

**Decide and write down first:**

- Endpoint list (Section 3) and what each returns.
- How unknown product groups and short history are reported.

**Proof it works:** Every endpoint works in /docs; deployed /health works.

## **dashboard/app.py** - Days 20-21

**Purpose:** Presentation only; calls the API.

**Before you touch it, be able to explain:**

- Streamlit reruns the whole script on each interaction; what session state is for.
- How to call an API and handle timeouts and a sleeping server.
- How a download button supplies a file.

**Decide and write down first:**

- Page sections; colours for Low, Medium, High; SIMULATED badge wording.
- API address read from a variable, not typed in.

**Proof it works:** Upload, forecast, risk, reorder and report download all work against the local API.

## **tests/test_*.py** - Days 22-23

**Purpose:** Automatic proof that important logic works.

**Before you touch it, be able to explain:**

- What a unit test is; arrange, act, assert.
- How pytest finds tests.
- How the FastAPI test client calls endpoints without starting a server.

**Decide and write down first:**

- List the cases: normal, invalid file, missing values, unknown product, short history, zero demand.

**Proof it works:** All tests pass; breaking a formula on purpose turns a test red.

## **README.md and docs** - Day 26

**Purpose:** How anyone installs, runs, tests and understands the project.

**Before you touch it, be able to explain:**

- Your architecture, data source and licence, evaluation results, assumptions, limitations.

**Decide and write down first:**

- Honest wording about simulated inventory and about baseline vs ML results.

**Proof it works:** A classmate follows the README and gets the dashboard running.

# 7. GitHub and hosting

## 7.1 GitHub habits

- Commit at the end of every day with a message describing what changed (for example "data cleaning handles missing dates").
- Before every push run **git status** and read what you are about to upload. Never commit .env, data/raw, passwords or connection strings.
- If you ever commit a secret by mistake, change that secret immediately (new database password); deleting the file later does not erase it from history.
- Keep the repository public if you want free Streamlit Community Cloud hosting (it deploys from GitHub). Make sure the README credits the dataset and states the licence.
- Tag the final version (for example v1.0) on Day 27 so you can always show exactly what you submitted.

## 7.2 Recommended hosting route

Verified on 23 October 2026 from provider pages. Free-tier terms change, so re-read them on Day 24 before relying on them.

| Part | Where | What I verified |
|---|---|---|
| FastAPI backend | Render free web service | Spins down after 15 minutes without traffic; the first request afterwards can take about a minute. Local files written at runtime are lost on restart or redeploy. |
| Database | Neon free Postgres | Free plan: 0.5 GB per project, compute scales to zero after 5 minutes of inactivity, no expiry listed on the plan page. |
| Dashboard | Streamlit Community Cloud | Deploys from a GitHub repo using a requirements file; secrets are set in its settings and read by the app. |
| Avoid | Render's free Postgres | Expires 30 days after creation, which could end before your defence. Use Neon instead. |

Other hosts exist (for example Hugging Face Spaces or PythonAnywhere). I have not verified their current free terms, so research them before choosing.

## 7.3 Deployment steps (Day 17 first deploy, Day 24 full deploy)

- **Neon:** create a project and database; copy its connection string; run your schema and loading steps against it from your laptop. Keep the string out of GitHub.
- **Render (API):** New Web Service from your GitHub repo. Build command installs requirements. Start command runs uvicorn on the app object, bound to 0.0.0.0 and the port Render provides in its PORT variable (the Render docs show the exact pattern; you will understand every part of it by Day 17). Add environment variables: DATABASE_URL and any others from config.py. Set the Python version to match your local version (see Render docs for how).
- **Model file:** because Render's disk is not permanent, commit the small saved model to the repo, or load from a place that persists. Do not train at startup.
- **Streamlit Community Cloud (dashboard):** New app, choose repo, branch, and dashboard/app.py. Put API_URL (your Render address) in the app's secrets. Give the dashboard its own small requirements file so it does not install the API's packages (check the Streamlit docs for where it looks for the file).
- **Cold starts:** the first request after idle may be slow. The dashboard should show "waking the server, please wait" and retry, and you should open the API a few minutes before your demo.

## 7.4 Environment variables table

| Variable | Used by | Where it lives |
|---|---|---|
| DATABASE_URL | API, loader scripts | Local .env (ignored by Git); Render environment settings |
| API_URL | Dashboard | Local .env; Streamlit Cloud secrets |

| Variable | Used by | Where it lives |
|---|---|---|
| MODEL_PATH (optional) | API | Defaults to the file in the repo |

## 7.5 Common deployment failures

| Symptom | Likely cause | How to check |
|---|---|---|
| ModuleNotFoundError in deploy logs | Package missing from requirements.txt | Compare imports with the file; reinstall in a clean venv locally. |
| App starts but is unreachable | Wrong start command, host or port | Read the start log lines; the server must listen on the port Render supplies. |
| Database connection refused or SSL error | Wrong or missing DATABASE_URL, SSL setting | Print only the host name (never the password) in logs; check Neon connection string. |
| Database URL rejected | URL prefix style the library does not accept | Read the library error; adjust the prefix as the docs describe. |
| File not found on Render but fine locally | Capital letters in file names (Linux is case-sensitive) or file ignored by .gitignore | Compare file names exactly; check git ls-files. |
| Model fails to load | Different library version than when it was saved | Match pinned versions; retrain and re-save locally. |
| Dashboard shows timeout | API sleeping | Wait one minute and retry; add the wake-up message. |

**Local demonstration fallback (prepare on Day 25)**

- Local PostgreSQL with data loaded, the API started with uvicorn, and the dashboard started with streamlit run - written as a 6-line checklist in the README.
- Screenshots of every dashboard screen and a short screen recording saved on your phone and laptop.
- If deployment fails during your defence, say so calmly and run the local version.

# 8. Testing, documentation and defence

## 8.1 What to test

- **Data:** valid file, missing columns, non-numeric values, negative values, duplicated dates, a gap in dates, empty file.
- **Forecasting:** baseline returns 7 values; short history falls back to the moving average; unknown product gives a clear error; no feature uses future data.
- **Inventory:** zero demand, zero stock, very long lead time, rounding to pack size, reorder never negative.
- **API:** health, each endpoint with good input, unknown product, bad horizon, invalid upload.
- **End to end:** upload data, see forecast, see risk, download report - run once locally and once deployed.

## 8.2 README checklist

- Project summary and the real-world problem.
- Data source, link, licence, and a clear statement of what is real and what is simulated.
- Architecture diagram and a short explanation.
- Setup and run instructions for Windows, from a clean machine.
- API endpoint list with example requests and responses.
- Evaluation: test period, metrics, baseline vs ML table, interpretation.
- Assumptions and limitations (ATC groups only, fractional volumes, single pharmacy, simulated stock).
- Deployment links and the local fallback steps.

## 8.3 Defence questions to practise (answer aloud, two minutes each)

1. Where did the data come from, and what are its limitations?
2. What exactly is a product in your data, and why do you call it a product group?
3. Why did you split the data chronologically rather than randomly?
4. What is your baseline and why is it a fair comparison?
5. What does your MAE mean in plain pharmacy terms?
6. Did the ML model beat the baseline? What does that tell you?
7. How did you avoid data leakage? Give one example feature.
8. How did you decide that a missing day is a zero rather than missing data?
9. Why is your inventory data simulated, and how do you prevent it being mistaken for real data?
10. Explain how safety stock and reorder point are calculated and what assumptions they rely on.
11. What is the difference between a forecast, a risk estimate and a recommendation?
12. What happens for a product with very little history?
13. Walk me through what happens when a user clicks Forecast.
14. Why did you choose Render, Neon and Streamlit Community Cloud, and what are their limits?
15. How do you keep secrets out of GitHub?
16. What would you need to make this safe for a real pharmacy?
17. What would you do with another month?

## 8.4 Seven-minute demo script (write your own version on Day 27)

- Minute 1: the problem and who it helps.
- Minute 2: the data, its source and honest limits.
- Minutes 3-4: live dashboard - upload, forecast, risk, reorder, report download.
- Minute 5: evaluation - baseline vs ML table and what you learned.
- Minute 6: architecture and testing in one slide or diagram.
- Minute 7: limitations, what is simulated, what you would improve.

# 9. Checklists and rescue rules

## End-of-week checklists

| Week | You may move on when |
|---|---|
| 1 | Raw files untouched; scope.md written; processed daily CSV exists; you can explain every cleaning decision; repo on GitHub without data or secrets. |
| 2 | Baseline and ML results in one table; one-group 7-day forecast runs; inventory functions match hand calculations. |
| 3 | Data in Postgres; all endpoints work; dashboard works locally from upload to report; health endpoint deployed. |
| 4 | Tests pass; deployed app works or fallback is ready; README complete; you can answer the 17 questions. |

## If you fall behind

- Behind by one day: use the Friday review day.
- Behind by two or more days at the end of Week 2: drop the ML model and ship baselines only, documenting why. A working complete system with a baseline beats an unfinished ML system.
- Behind at the end of Week 3: drop the PDF/extra charts; keep the CSV report; reduce the database to products, daily_sales and recommendations.
- Deployment failing on 18 Nov: stop troubleshooting; finish documentation and prepare the local demo.

## Official links to keep (verify before relying on any plan limits)

- Dataset: kaggle.com/datasets/milanzdravkovic/pharma-sales-data
- Render free tier: render.com/docs/free
- Neon pricing: neon.com/pricing
- Streamlit deployment docs: docs.streamlit.io/streamlit-cloud/get-started/deploy-an-app
- FastAPI docs: fastapi.tiangolo.com
- Streamlit docs: docs.streamlit.io
- pandas docs: pandas.pydata.org
- scikit-learn docs: scikit-learn.org

> **When you are stuck**
>
> Come back with: what you were trying to do, the exact error text or output, and what you already tried. I will explain the error, point to the likely cause, and give the smallest hint first. That is how you keep ownership of the project.